

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación · Ingeniería de Software

Desarrollo de Aplicaciones Web con PERL

Guía del estudiante — Unidad II: Programación del lado del servidor

De la instalación al despliegue de una aplicación CRUD con PostgreSQL

Asignatura: Aplicaciones Web (5.º semestre)

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Período: PPA 2026–2027

Contenido

1	Introducción	4
1.1	¿Qué es PERL y dónde encaja en la web?	4
1.2	De CGI a los frameworks modernos	4
1.3	Qué aprenderás	4
2	Preparación del entorno de desarrollo	5
2.1	Strawberry Perl	5
2.2	IntelliJ IDEA y el plugin Perl	5
2.3	Configurar el intérprete (perl.exe)	5
2.4	Estructura del proyecto	5
2.5	Instalar módulos con CPAN	6
2.6	Nota sobre redes institucionales (inspección SSL)	6
2.7	Versiones de referencia (verificadas a junio de 2026)	6
3	Fundamentos de PERL para la web	6
3.0.1	Anatomía de un script	7
3.0.2	Variables y sigilos	7
3.0.3	Comillas simples vs dobles (interpolación)	7
3.0.4	Operadores y símbolos esenciales	7
3.0.5	Expresiones regulares	8
3.0.6	Heredocs	8
3.0.7	Subrutinas	8
3.0.8	Tabla de referencia rápida	9
4	El modelo de la web del lado del servidor	9
4.1	Request-response y HTTP sin estado	9
4.2	La respuesta HTTP	10
5	Primera aplicación web: «Hola Mundo + nombre»	10
5.1	Código completo (hola_web.pl)	11
5.2	Cómo funciona, en breve	13
5.3	Ejecutar y probar	14
5.4	Las dos reglas de oro	14

6	Generar HTML5 semántico desde PERL	14
6.1	Elementos semánticos de estructura	14
6.2	Características modernas de HTML5	14
6.3	Separar lógica de presentación	14
7	Acceso a datos con PostgreSQL	15
7.1	DBI y el driver DBD::Pg	15
7.2	Datos de conexión y DSN	15
7.3	Crear las tablas (schema.sql)	15
7.4	El ciclo de DBI	16
7.5	Programa de ejemplo (bd_postgres.pl)	17
7.6	¿Y un ORM?	18
8	Aplicación web completa con base de datos (CRUD)	18
8.1	Qué hace	18
8.2	Código (crud_estudiantes.pl)	19
8.3	Estructura y flujo	25
8.4	Datos de prueba	25
9	Aplicación con ORM: DBIx::Class (paso a paso)	26
9.1	¿Cuándo usar un ORM?	26
9.2	Paso 1: Instalar los módulos	26
9.3	Paso 2: La estructura de archivos	26
9.4	Paso 3: La clase Schema	27
9.5	Paso 4: Las clases Result (el modelo)	27
9.6	Paso 5: Conectar y crear las tablas con el ORM	28
9.7	Paso 6: Operaciones CRUD con el ORM	29
9.8	Paso 7: Integrarlo en la aplicación web	30
9.9	Paso 8: Ejecutar y resultado	31
10	Seguridad en aplicaciones web	31
11	Buenas prácticas y mantenimiento	31
12	Solución de problemas comunes	32

13	Glosario de siglas y acrónimos	32
14	Ejercicios propuestos	33
15	Recursos y referencias	33

1 Introducción

1.1 ¿Qué es PERL y dónde encaja en la web?

PERL (Practical Extraction and Report Language) es un lenguaje creado por Larry Wall en 1987. Es maduro, pragmático y especialmente potente para el manejo de texto y las expresiones regulares. Durante los años 90 fue el lenguaje que hizo dinámica a la web: combinado con CGI (Common Gateway Interface), permitió por primera vez procesar formularios y generar páginas en el servidor.

Hoy PERL convive con PHP, Java y .NET. Se usa menos para aplicaciones nuevas de gran escala, pero sigue vigente en administración de sistemas, automatización e integración de datos. Estudiarlo te da una perspectiva valiosa: comprenderás el modelo cliente-servidor desde sus cimientos, sin la «magia» que ocultan los frameworks.

Su mayor activo es **CPAN** (Comprehensive Perl Archive Network): un enorme repositorio con un módulo para casi cualquier necesidad, incluido el acceso a bases de datos y a servidores web.

1.2 De CGI a los frameworks modernos

Existen varias formas de servir una aplicación web escrita en PERL:

- **CGI clásico:** el servidor web (Apache) ejecuta un script por cada petición. Es la forma histórica; hoy se considera legado, útil para aprender el modelo.
- **Servidor propio:** con el módulo del núcleo `IO::Socket::INET` se levanta un mini servidor HTTP sin instalar Apache. Ideal para ejercicios y prototipos.
- **PSGI/Plack:** el estándar moderno que desacopla la aplicación del servidor (equivale al WSGI de Python). Se ejecuta con `plackup`.
- **Frameworks:** para proyectos serios existen Mojolicious, Dancer2 y Catalyst, que aportan enrutamiento, plantillas y utilidades sobre PSGI/Plack.

En esta guía usaremos un servidor propio con `IO::Socket::INET` porque funciona de inmediato en cualquier máquina y deja ver con claridad el ciclo petición → respuesta.

1.3 Qué aprenderás

- Preparar el entorno (Strawberry Perl e IntelliJ IDEA) para programar en PERL.
- Comprender la sintaxis esencial de PERL aplicada a la web.
- Construir una aplicación web que reciba datos del usuario y genere HTML5 semántico.
- Acceder a una base de datos PostgreSQL con DBI y sentencias preparadas.
- Desarrollar una aplicación CRUD completa (alta, consulta/reporte y eliminación).
- Aplicar las prácticas de seguridad fundamentales del desarrollo web.

2 Preparación del entorno de desarrollo

2.1 Strawberry Perl

En Windows, la distribución recomendada es Strawberry Perl (versión actual: **5.42.2.1**, que incluye Perl 5.42). Incluye el intérprete, un compilador (gcc) y un amplio conjunto de módulos preinstalados. Descárgalo de strawberryperl.com y acepta la ruta por defecto **C:\Strawberry**.

Tras instalar, verifica en una terminal (cmd):

Terminal

```
1 C:\> perl -v
2
3 This is perl 5, version 42, subversion 1 (v5.42.1) ...
4 built for MSWin32-x64
```

Importante

Strawberry incluye el módulo DBI, pero NO el driver de PostgreSQL DBD::Pg: ese se instala aparte (sección 2.5).

2.2 IntelliJ IDEA y el plugin Perl

IntelliJ IDEA es el entorno de desarrollo (IDE). Para que entienda PERL hay que instalar el plugin «**Perl**» (conocido técnicamente como Camelcade), que añade resaltado de sintaxis, autocompletado, ejecución y depuración. Se instala desde Settings → Plugins → Marketplace.

2.3 Configurar el intérprete (perl.exe)

Al crear un proyecto, IntelliJ pide el **Perl5 Interpreter**. Debes apuntarlo al archivo **perl.exe**, que con la instalación por defecto está en:

Ruta del intérprete

```
1 C:\Strawberry\perl\bin\perl.exe
```

Cuidado

No apuntes a **wperl.exe** (variante sin consola, no sirve para ejecutar/depurar) ni a la carpeta **c\bin** (ahí está el compilador gcc, no el intérprete).

2.4 Estructura del proyecto

Un proyecto Perl en IntelliJ crea dos carpetas: **lib** (para módulos reutilizables, archivos **.pm**) y **t** (para pruebas). El script ejecutable principal puede ir en la raíz del proyecto.

2.5 Instalar módulos con CPAN

Los módulos que no vienen incluidos se instalan desde la terminal con `cpan` o `cpanm`. Por ejemplo, el driver de PostgreSQL:

Instalar el driver

```
1 C:\> cpan DBD::Pg
```

2.6 Nota sobre redes institucionales (inspección SSL)

En la red de la UTEQ, al conectar con servicios externos puede aparecer un aviso de «Untrusted Server's Certificate» firmado por un firewall (Hillstone). No es un error tuyo: es la inspección SSL/TLS de la red institucional. En esa red puedes pulsar «Save as Trusted» para continuar; alternatively, descarga el plugin (.zip) e instálalo con Settings → Plugins → Install Plugin from Disk.

2.7 Versiones de referencia (verificadas a junio de 2026)

Estas son las versiones vigentes de las herramientas usadas en esta guía. Los módulos de Perl se instalan con `cpan`, que siempre descarga la última versión disponible, así que no es necesario fijarlas a mano.

Componente	Versión vigente (jun. 2026)
Perl	5.42.1 (rama estable; las 5.43.x son de desarrollo).
Strawberry Perl	5.42.2.1 (incluye Perl 5.42, gcc y módulos).
PostgreSQL	18.4 (serie 18; la 19 está en beta para finales de 2026).
DBI	Incluido en Strawberry; última disponible en CPAN.
DBD::Pg	3.20.2 (última en CPAN).
DBIx::Class	Última disponible en CPAN (serie 0.0828x).
IntelliJ IDEA	2026.1.3 (Community o Ultimate).
Plugin Perl	Camelcade, última compatible con tu IntelliJ.

3 Fundamentos de PERL para la web

Esta sección resume la sintaxis mínima que necesitarás para entender los programas de la guía.

3.0.1 Anatomía de un script

Todo script Perl profesional empieza con la línea «shebang» y dos directivas de disciplina:

Cabecera de un script

```
1 #!/usr/bin/perl          # interprete (Linux/Mac); en Windows se ignora
2 use strict;              # obliga a declarar variables con 'my'
3 use warnings;            # avisa de practicas peligrosas
4 use utf8;                # el archivo fuente esta en UTF-8
```

El símbolo # inicia un comentario: Perl ignora todo lo que sigue hasta el fin de la línea.

3.0.2 Variables y sigilos

PERL marca el tipo de variable con un símbolo inicial llamado «sigilo»:

- \$ escalar: un solo valor (número o texto). Ej.: `my $nombre = 'Ana';`
- @ arreglo: una lista ordenada. Ej.: `my @datos = (1, 2, 3);`
- % hash: pares clave => valor. Ej.: `my %f = (nombre => 'Ana');`

3.0.3 Comillas simples vs dobles (interpolación)

Es una de las distinciones más importantes y fuente de errores frecuentes:

- **Comillas dobles** interpolan: las variables se sustituyen por su valor y se procesan secuencias como salto de línea.
- **Comillas simples** son literales: el texto se toma tal cual.

Caso real: la contraseña de la base de datos

La clave `P@$7Gr3$QL` contiene \$ y @. Debe escribirse en comillas SIMPLES:

```
my $clave = 'P@$7Gr3$QL';
```

En comillas dobles, Perl tomaría \$7 y \$QL como variables y la clave llegaría incompleta.

3.0.4 Operadores y símbolos esenciales

Símbolo	Significado
<code>= ;</code>	Asignación de valor; fin de toda instrucción.
<code>-></code>	Operador flecha: llama a un método de un objeto (p. ej. <code>\$dbh->prepare</code>).
<code>=></code>	«Coma gorda»: une una opción con su valor y entrecomilla el nombre de la izquierda.

Símbolo	Significado
<code>=~</code>	Enlaza un texto con una expresión regular.
<code>.</code>	Concatena (une) cadenas de texto.
<code>//=</code>	Asigna un valor por defecto solo si la variable está indefinida.
<code>or die</code>	O lógico de baja prioridad; «die» aborta el programa con un mensaje.

3.0.5 Expresiones regulares

Permiten buscar y transformar texto. Se usan con `=~` y estos operadores:

- `m{...}` coincidencia (match). Lo que capturan los paréntesis queda en `$1`.
- `s/buscar/reemplazo/g` sustitución. El modificador `g` aplica a todas; `e` evalúa el reemplazo como código.
- `tr/a/b/` transliteración: cambia caracteres uno a uno.

En los patrones: `()` captura, `[...]` clase de caracteres, `[^...]` clase negada, `\s` espacio, `\d` dígito, `*` cero o más, `{2}` exactamente dos.

3.0.6 Heredocs

Un «heredoc» guarda un bloque largo de texto (por ejemplo, una página HTML) en una variable. Con comillas dobles en el marcador, interpola variables:

Heredoc

```
1 my $html = <<"HTML";
2 <h1>Hola, $nombre</h1>
3 HTML
```

Truco importante del heredoc

Como interpola, una `@` o `$` literal dentro del texto debe escaparse con barra invertida: `\@media` en CSS, o `correo\@uteq.edu.ec`. Si no, Perl las interpreta como variables.

3.0.7 Subrutinas

Una función se define con `sub`. Los argumentos llegan en el arreglo `@_` y se devuelve un valor con `return`:

Subrutina

```
1 sub saludar {  
2     my ($nombre) = @_;      # primer argumento  
3     return "Hola, $nombre";  
4 }
```

3.0.8 Tabla de referencia rápida

Funciones y palabras clave que aparecen en los programas de esta guía:

Función	Para qué sirve
<code>use</code>	Carga un módulo o una directiva (pragma).
<code>my</code>	Declara una variable local.
<code>if / unless</code>	Ejecuta si la condición es verdadera / falsa.
<code>while</code>	Repite mientras la condición sea verdadera.
<code>defined</code>	Indica si una variable tiene valor.
<code>print</code>	Escribe texto (en consola o en un socket).
<code>length</code>	Número de caracteres de una cadena.
<code>chr / hex</code>	Carácter de un código / hexadecimal a número.
<code>sub / return</code>	Define una función / devuelve su resultado.
<code>die</code>	Aborta el programa con un mensaje de error.
<code>close</code>	Cierra una conexión, socket o archivo.

4 El modelo de la web del lado del servidor

4.1 Request-response y HTTP sin estado

La web funciona con un modelo petición–respuesta (request–response): el navegador (cliente) envía una petición HTTP y el servidor devuelve una respuesta. El servidor de aplicaciones ejecuta código (PERL) que genera dinámicamente esa respuesta según la petición, la sesión y la base de datos.

Un punto clave: HTTP es **sin estado** (stateless). El servidor no recuerda al usuario entre una petición y la siguiente; por eso, para «recordarlo», se usan cookies y sesiones (tema de la Unidad

III).

4.2 La respuesta HTTP

Una respuesta HTTP tiene un orden estricto: una línea de estado, las cabeceras, una línea en blanco y el cuerpo:

Respuesta HTTP

```
1 HTTP/1.1 200 OK
2 Content-Type: text/html; charset=utf-8
3 Content-Length: 123
4
5 <!DOCTYPE html> ... </html>
```

El código 200 significa «todo correcto»; 302 es una redirección; 404 no encontrado. La cabecera `Content-Type` declara que es HTML en UTF-8, esencial para que las tildes se vean bien.

5 Primera aplicación web: «Hola Mundo + nombre»

Esta primera aplicación es un mini servidor web que saluda al usuario con el nombre que escriba en un formulario. No necesita Apache: usa solo módulos del núcleo de Perl.

5.1 Código completo (hola_web.pl)

hola_web.pl — parte 1/4

```

1  #!/usr/bin/perl
2  # =====
3  #  hola_web.pl  -  MiHolaMundo
4  #  UTEQ - Aplicaciones Web - Unidad II, Semana 5
5  #
6  #  Aplicacion web "Hola Mundo + nombre". Es un mini servidor web que NO necesita
7  #  Apache ni modulos de CPAN: usa solo modulos del nucleo de Perl, asi que
8  #  funciona tal cual en Strawberry Perl.
9  #
10 #  COMO EJECUTARLO EN INTELLIJ IDEA
11 #  -----
12 #  1) Clic derecho sobre el proyecto MiHolaMundo > New > File -> hola_web.pl
13 #  2) Pega este contenido.
14 #  3) Pulsa el boton verde Run (o Shift+F10).
15 #  4) En la consola aparece: Servidor escuchando en http://localhost:8080
16 #  5) Abre esa direccion en el navegador, escribe un nombre y pulsa "Saludar".
17 #  6) Para detenerlo: boton rojo Stop en IntelliJ.
18 #  =====
19
20 use strict;                # obliga a declarar variables (evita errores tontos)
21 use warnings;              # avisa de practicas peligrosas
22 use IO::Socket::INET;      # modulo del NUCLEO de Perl: sockets de red
23
24 my $puerto = 8080;
25
26 # Crea el socket servidor que se queda escuchando conexiones entrantes.
27 my $servidor = IO::Socket::INET->new(
28     LocalAddr => '127.0.0.1',    # solo este equipo (localhost)
29     LocalPort => $puerto,
30     Proto      => 'tcp',
31     Listen     => 5,
32     ReuseAddr  => 1,
33 ) or die "No se pudo abrir el puerto $puerto: $!\n";
34
35 print "Servidor escuchando en http://localhost:$puerto\n";
36 print "(pulsa el boton rojo Stop en IntelliJ para detenerlo)\n";
37
38 # Bucle principal: atiende una conexion del navegador tras otra.
39 while (my $cliente = $servidor->accept) {

```

hola_web.pl — parte 2/4

```

1  # Lee la primera línea de la petición HTTP. Ejemplo:
2  #   GET /?nombre=Ana HTTP/1.1
3  my $peticion = <$cliente>;
4  $peticion = '' unless defined $peticion;
5
6  # --- LECTURA SEGURA DE LA ENTRADA -----
7  # Regla de oro del backend: nunca confiar en lo que envía el usuario.
8  my $nombre = 'Mundo';           # valor por defecto
9  if ($peticion =~ m{[?&]nombre=([^\s&]*)}) { # busca ?nombre=...
10     $nombre = url_decode($1);
11 }
12 $nombre = escape_html($nombre);    # escapa la salida -> anti-XSS
13
14 # --- CUERPO HTML5 -----
15 # Heredoc con comillas dobles: Perl interpola $nombre dentro del texto.
16 my $html = <<"HTML";
17 <!DOCTYPE html>
18 <html lang="es-EC">
19 <head>
20   <meta charset="UTF-8">
21   <meta name="viewport" content="width=device-width, initial-scale=1">
22   <title>Hola Mundo en PERL - UTEQ</title>
23   <style>
24     body { font-family: system-ui, sans-serif; max-width: 36rem;
25           margin: 3rem auto; padding: 0 1rem; color: #1f2933; line-height: 1.6; }
26     h1 { color: #006633; }
27     form { margin-top: 1.5rem; display: flex; gap: .6rem; }
28     input[type=text] { flex: 1; padding: .6rem; font-size: 1rem;
29                     border: 1px solid #bbb; border-radius: .4rem; }
30     button { padding: .6rem 1.2rem; font-size: 1rem; cursor: pointer; border: 0;
31             background: #006633; color: #fff; border-radius: .4rem; }
32     small { color: #6b7280; }
33   </style>
34 </head>
35 <body>
36   <h1>&iexcl;Hola Mundo, $nombre!</h1>
37   <p>Esta página genera un programa <strong>PERL</strong> en el servidor,
38     no es un archivo HTML estatico.</p>
39
40   <!-- Formulario GET: al enviar recarga la página con  ?nombre=loquescribas -->

```

hola_web.pl — parte 3/4

```

1  <form method="get" action="/">
2    <input type="text" name="nombre" placeholder="Escribe tu nombre" autofocus>
3    <button type="submit">Saludar</button>
4  </form>
5
6  <p><small>Servido por PERL en http://localhost:$puerto</small></p>
7 </body>
8 </html>
9 HTML
10
11 # --- RESPUESTA HTTP -----
12 # Siempre: línea de estado + cabeceras + línea en blanco + cuerpo.
13 my $bytes = length $html;
14 print $cliente "HTTP/1.1 200 OK\r\n";
15 print $cliente "Content-Type: text/html; charset=utf-8\r\n";
16 print $cliente "Content-Length: $bytes\r\n";
17 print $cliente "Connection: close\r\n";
18 print $cliente "\r\n";
19 print $cliente $html;
20
21 close $cliente;
22 }
23
24 # === Funciones auxiliares =====
25
26 # Decodifica una query string: '+' -> espacio, '%XX' -> su caracter.
27 sub url_decode {
28     my ($s) = @_;
29     $s =~ tr/+// /;
30     $s =~ s/%([0-9A-Fa-f]{2})/chr(hex($1))/ge;
31     return $s;
32 }
33
34 # Escapa los caracteres peligrosos para evitar ataques XSS al imprimir en HTML.
35 sub escape_html {
36     my ($s) = @_;
37     $s =~ s/&/&amp;/g;
38     $s =~ s/</&lt;/g;
39     $s =~ s/>/&gt;/g;
40     $s =~ s/"/&quot;/g;

```

hola_web.pl — parte 4/4

```

1     return $s;
2 }

```

5.2 Cómo funciona, en breve

1. Abre un socket servidor en el puerto 8080 con `IO::Socket::INET`.
2. Por cada visita, lee la primera línea de la petición y extrae el parámetro `nombre` de la URL con una expresión regular.

3. Valida y **escapa** ese valor (defensa anti-XSS) antes de mostrarlo.
4. Construye una página HTML5 con un heredoc y la envía con las cabeceras HTTP correctas.

5.3 Ejecutar y probar

Pulsa el botón verde Run en IntelliJ. En la consola aparece **Servidor escuchando en `http://localhost:8080`**. El programa queda «escuchando» (es lo normal; se detiene con el botón Stop). Abre esa dirección en el navegador, escribe un nombre y pulsa «Saludar».

5.4 Las dos reglas de oro

Seguridad del lado del servidor

- 1) **Validar TODA entrada en el servidor:** el cliente es manipulable (las DevTools del navegador permiten saltar cualquier validación de HTML/JS).
- 2) **Escapar TODA salida:** convertir `<` `>` `&` comillas en entidades evita que un `<script>` malicioso se ejecute (ataque XSS).

6 Generar HTML5 semántico desde PERL

El HTML que genera el servidor debe ser semántico: usar etiquetas que describan el significado de cada parte, no solo su apariencia. Esto mejora la accesibilidad, el posicionamiento en buscadores (SEO) y el mantenimiento.

6.1 Elementos semánticos de estructura

Úsalos para organizar la página: `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>` y `<address>`.

6.2 Características modernas de HTML5

- `<time datetime="...">` fechas legibles por máquinas; `<mark>` resaltado.
- `<details>/<summary>` paneles desplegables nativos (sin JavaScript).
- `<meter>` y `<output>` indicadores y resultados.
- Validación nativa de formularios: `required`, `minlength`, `type=email`, `type=search`.

6.3 Separar lógica de presentación

Aunque aquí generamos el HTML con heredocs por simplicidad, en proyectos serios conviene separar la lógica (PERL) de la presentación usando plantillas (por ejemplo, Template Toolkit). Así el código no se mezcla con el diseño y es más fácil de mantener.

7 Acceso a datos con PostgreSQL

7.1 DBI y el driver DBD::Pg

PERL accede a bases de datos a través de **DBI** (Database Interface), una API común para cualquier base. El driver concreto para PostgreSQL es **DBD::Pg**, que se instala con `cpan DBD::Pg` y requiere PostgreSQL instalado.

7.2 Datos de conexión y DSN

El DSN (Data Source Name) describe la conexión:

Dato	Valor
Base	appweb2026ppa
Host	127.0.0.1 (localhost)
Puerto	5432
Usuario	postgres
Clave	P@/\$7Gr3/\$QL (en comillas simples)

Construcción del DSN

```
1 my $dsn = "dbi:Pg:dbname=appweb2026ppa;host=127.0.0.1;port=5432";
```

7.3 Crear las tablas (schema.sql)

El esquema de la base se define con SQL. Puedes ejecutarlo en `psql` o `pgAdmin`; además, la aplicación lo crea automáticamente si no existe:

schema.sql

```

1  -- =====
2  --  schema.sql  -  UTEQ - Aplicaciones Web - Unidad II
3  --  Crea la tabla que usa la aplicacion web de registro de estudiantes.
4  --
5  --  Como ejecutarlo:
6  --      psql -U postgres -d appweb2026ppa -f schema.sql
7  --  o pegarlo en el editor de consultas de pgAdmin (conectado a appweb2026ppa).
8  --
9  --  Nota: la aplicacion crud_estudiantes.pl tambien ejecuta este CREATE TABLE
10 --  IF NOT EXISTS al arrancar, asi que correr este archivo es opcional pero
11 --  recomendable para tener el esquema versionado.
12 -- =====
13
14 CREATE TABLE IF NOT EXISTS estudiantes (
15     id         SERIAL         PRIMARY KEY,           -- identificador autoincremental
16     nombre     VARCHAR(100) NOT NULL,               -- nombre completo
17     correo     VARCHAR(120) NOT NULL,               -- correo electronico
18     carrera    VARCHAR(80)  NOT NULL,               -- carrera a la que pertenece
19     creado     TIMESTAMP      NOT NULL DEFAULT NOW() -- fecha/hora de registro
20 );
21
22 -- Indice opcional para acelerar reportes por carrera:
23 CREATE INDEX IF NOT EXISTS idx_estudiantes_carrera ON estudiantes (carrera);

```

7.4 El ciclo de DBI

Método	Qué hace
<code>DBI->connect</code>	Abre la conexión; devuelve el database handle (\$dbh).
<code>\$dbh->do</code>	Ejecuta SQL que no devuelve filas (CREATE, INSERT...).
<code>\$dbh->prepare</code>	Prepara una sentencia y devuelve un statement handle (\$sth).
<code>\$sth->execute</code>	Ejecuta la sentencia (con o sin valores para los ?).
<code>fetchrow_hashref</code>	Lee la siguiente fila como hash {columna => valor}.
<code>finish/disconnect</code>	Cierra la sentencia / la conexión.

Regla de oro de datos: sentencias preparadas

Usa siempre marcadores ? y `execute(valores)`. Nunca concatenes datos del usuario dentro del SQL: es la defensa contra la inyección SQL (SQL Injection).

7.5 Programa de ejemplo (bd_postgres.pl)

Conecta, crea la tabla, inserta filas con sentencias preparadas y consulta los datos:

bd_postgres.pl — parte 1/2

```

1  #!/usr/bin/perl
2  # =====
3  #  bd_postgres.pl  -  UTEQ - Aplicaciones Web - Unidad II
4  #  Acceso a una base de datos PostgreSQL desde PERL con DBI + DBD::Pg.
5  #  Conecta, crea una tabla, inserta filas con sentencias preparadas,
6  #  consulta los datos y los muestra. Al final cierra la conexion.
7  #
8  #  REQUISITOS (una sola vez, en la terminal):
9  #      cpan DBI          # interfaz de bases de datos (Strawberry suele traerlo)
10 #      cpan DBD::Pg      # driver de PostgreSQL (NO viene por defecto)
11 #  Ademas: PostgreSQL en ejecucion y la base 'appweb2026ppa' creada.
12 #  =====
13
14 use strict;
15 use warnings;
16 use utf8;
17 use DBI;                                # interfaz estandar de bases de datos en Perl
18
19 binmode(STDOUT, ':encoding(UTF-8)');
20
21 # --- Datos de conexion -----
22 my $bd      = 'appweb2026ppa';          # nombre de la base de datos
23 my $host     = '127.0.0.1';             # servidor (este equipo)
24 my $puerto  = 5432;                   # puerto de PostgreSQL
25 my $usuario  = 'postgres';              # usuario
26
27 # OJO: la clave lleva $ y @, por eso va en COMILLAS SIMPLES (no interpola).
28 my $clave    = 'P@57Gr3$QL';
29
30 # DSN (Data Source Name): cadena que describe la conexion.
31 my $dsn = "dbi:Pg:dbname=$bd;host=$host;port=$puerto";
32
33 # --- Conectar -----
34 my $dbh = DBI->connect($dsn, $usuario, $clave, {
35     RaiseError    => 1,    # si algo falla, lanza error y aborta
36     AutoCommit    => 1,    # confirma cada sentencia automaticamente
37     pg_enable_utf8 => 1,    # maneja el texto como UTF-8
38 }) or die "No se pudo conectar: $DBI::errstr";
39
40 print "Conexion exitosa a la base '$bd'.\n";

```

bd_postgres.pl — parte 2/2

```

1
2 # --- Crear la tabla (si no existe) -----
3 $dbh->do(q{
4     CREATE TABLE IF NOT EXISTS estudiantes (
5         id      SERIAL PRIMARY KEY,
6         nombre  VARCHAR(100) NOT NULL,
7         correo  VARCHAR(120)
8     )
9 });
10
11 # --- Insertar con SENTENCIA PREPARADA (anti SQL Injection) -----
12 my $insertar = $dbh->prepare(
13     'INSERT INTO estudiantes (nombre, correo) VALUES (?, ?)'
14 );
15 $insertar->execute('Ana Perez', 'ana@uteq.edu.ec');
16 $insertar->execute('Luis Mora', 'luis@uteq.edu.ec');
17
18 # --- Consultar -----
19 my $consulta = $dbh->prepare(
20     'SELECT id, nombre, correo FROM estudiantes ORDER BY id'
21 );
22 $consulta->execute;
23
24 print "\nEstudiantes registrados:\n";
25 while (my $fila = $consulta->fetchrow_hashref) {
26     print "  [$fila->{id}] $fila->{nombre} - $fila->{correo}\n";
27 }
28
29 # --- Cerrar -----
30 $consulta->finish;
31 $dbh->disconnect;
32 print "\nConexion cerrada.\n";

```

7.6 ¿Y un ORM?

PERL sí tiene ORM (mapeo objeto-relacional): el más completo es **DBIx::Class**, que incluso puede crear las tablas desde el modelo. Sin embargo, es pesado y requiere definir clases de esquema; suele introducirse en proyectos grandes. Para aprender y para aplicaciones pequeñas, DBI con sentencias preparadas es más directo y transparente. (En el Proyecto Fin de Curso usarás un ORM, JPA, pero en Java.) Lo desarrollamos paso a paso en la sección dedicada al ORM.

8 Aplicación web completa con base de datos (CRUD)

8.1 Qué hace

Reúne todo lo anterior en una aplicación interactiva de registro de estudiantes:

- **Alta:** un formulario inserta estudiantes (nombre, correo, carrera).
- **Reporte:** una tabla con todos los registros y un resumen por carrera (GROUP BY).

- **Eliminar:** un enlace por fila borra el registro.
- **Patrón Post-Redirect-Get:** tras un envío POST, redirige (302) para que recargar no reenvíe el formulario.

8.2 Código (crud_estudiantes.pl)

Para mantener el listado legible se ha colapsado el bloque de estilos CSS; el archivo completo se entrega aparte.

crud_estudiantes.pl — parte 1/7

```

1  #!/usr/bin/perl
2  # =====
3  #  crud_estudiantes.pl  -  UTEQ - Aplicaciones Web - Unidad II
4  #  Aplicacion WEB que interactua con el usuario y usa PostgreSQL:
5  #      - Formulario para REGISTRAR estudiantes (alta de datos).
6  #      - REPORTE: tabla con todos los estudiantes + resumen por carrera.
7  #      - Accion para ELIMINAR un estudiante.
8  #  Es un mini servidor web en Perl puro (sin Apache) que accede a la base
9  #  con DBI + sentencias preparadas (seguras contra SQL Injection).
10 #
11 #  REQUISITOS (una sola vez):
12 #      cpan DBD::Pg          # driver de PostgreSQL (DBI ya viene con Strawberry)
13 #      PostgreSQL en marcha y la base 'appweb2026ppa' creada.
14 #
15 #  EJECUTAR: boton verde Run en IntelliJ; luego abrir http://localhost:8080
16 #  =====
17
18 use strict;
19 use warnings;
20 use utf8;
21 use IO::Socket::INET;      # mini servidor web (modulo del nucleo)
22 use DBI;                   # acceso a la base de datos
23
24 binmode(STDOUT, ':encoding(UTF-8)');
25
26 # -----
27 # 1) CONEXION A POSTGRESQL
28 # -----
29 my $bd      = 'appweb2026ppa';
30 my $host     = '127.0.0.1';
31 my $puerto_bd = 5432;
32 my $usuario  = 'postgres';
33 my $clave    = 'P@$7Gr3$QL'; # comillas SIMPLES por el $ y la @
34 my $dsn = "dbi:Pg:dbname=$bd;host=$host;port=$puerto_bd";
35
36 my $dbh = DBI->connect($dsn, $usuario, $clave, {
37     RaiseError      => 1,
38     AutoCommit      => 1,
39     pg_enable_utf8 => 1,
40 }) or die "No se pudo conectar a la base: $DBI::errstr";

```

crud_estudiantes.pl — parte 2/7

```

1
2 # -----
3 # 2) CREAR LA TABLA SI NO EXISTE (mismo esquema que schema.sql)
4 # -----
5 $dbh->do(q{
6     CREATE TABLE IF NOT EXISTS estudiantes (
7         id      SERIAL      PRIMARY KEY,
8         nombre  VARCHAR(100) NOT NULL,
9         correo  VARCHAR(120) NOT NULL,
10        carrera VARCHAR(80)  NOT NULL,
11        creado  TIMESTAMP    NOT NULL DEFAULT NOW()
12    )
13 });
14
15 # -----
16 # 3) SERVIDOR WEB
17 # -----
18 my $puerto_web = 8080;
19 my $servidor = IO::Socket::INET->new(
20     LocalAddr => '127.0.0.1',
21     LocalPort => $puerto_web,
22     Proto     => 'tcp',
23     Listen    => 5,
24     ReuseAddr => 1,
25 ) or die "No se pudo abrir el puerto $puerto_web: $!\n";
26
27 print "Servidor en http://localhost:$puerto_web (pulsa Stop para detener)\n";
28
29 while (my $cli = $servidor->accept) {
30
31     # --- Leer la linea de peticion: p.ej. POST /agregar HTTP/1.1 ---
32     my $inicial = <$cli>;
33     unless (defined $inicial) { close $cli; next; }
34     my ($metodo, $ruta) = $inicial =~ m{^\s+)\s+(\s+)};
35     $metodo //= 'GET';
36     $ruta    //= '/';
37
38     # --- Leer cabeceras hasta la linea en blanco; tomar Content-Length ---
39     my $largo = 0;
40     while (my $h = <$cli>) {

```

crud_estudiantes.pl — parte 3/7

```

1      $h =~ s/\r?\n\z//;
2      last if $h eq '';
3      $largo = $1 if $h =~ /^Content-Length:\s*(\d+)/i;
4  }
5
6  # --- Leer el cuerpo (datos del formulario en POST) ---
7  my $cuerpo = '';
8  read($cli, $cuerpo, $largo) if $largo > 0;
9
10 # --- Enrutamiento ---
11 if ($metodo eq 'POST' && $ruta =~ m{~/agregar}) {
12     my %f = parse_form($cuerpo);
13     # Validacion minima en el servidor (nunca confiar en el cliente)
14     if (length($f{nombre} // '') && length($f{correo} // '') && length($f{carrera} // '')) {
15         my $ins = $dbh->prepare(
16             'INSERT INTO estudiantes (nombre, correo, carrera) VALUES (?, ?, ?)');
17         $ins->execute($f{nombre}, $f{correo}, $f{carrera});
18     }
19     redirigir($cli, '/');          # Patron PRG (Post-Redirect-Get)
20 }
21 elsif ($metodo eq 'GET' && $ruta =~ m{~/eliminar\?id=(\d+)}) {
22     my $id = $1;
23     my $del = $dbh->prepare('DELETE FROM estudiantes WHERE id = ?');
24     $del->execute($id);
25     redirigir($cli, '/');
26 }
27 else {
28     responder($cli, pagina());      # GET / : formulario + reporte
29 }
30
31 close $cli;
32 }
33
34 $dbh->disconnect;
35
36 # =====
37 #  SUBROUTINAS
38 # =====
39
40 # Convierte el cuerpo "a=1&b=2" de un formulario en un hash de Perl.

```

crud_estudiantes.pl — parte 4/7

```

1 sub parse_form {
2     my ($cuerpo) = @_;
3     my %datos;
4     for my $par (split /\&/, $cuerpo) {
5         my ($k, $v) = split /=/, $par, 2;
6         next unless defined $k;
7         $datos{ url_decode($k) } = url_decode($v // '');
8     }
9     return %datos;
10 }
11
12 # Construye la pagina HTML5 con el formulario y los reportes.
13 sub pagina {
14     # --- Reporte 1: todos los estudiantes ---
15     my $sth = $dbh->prepare(
16         'SELECT id, nombre, correo, carrera FROM estudiantes ORDER BY id');
17     $sth->execute;
18
19     my $filas = '';
20     my $total = 0;
21     while (my $e = $sth->fetchrow_hashref) {
22         $total++;
23         $filas .=
24             "<tr>"
25             . "<td>" . escape_html($e->{id}) . "</td>"
26             . "<td>" . escape_html($e->{nombre}) . "</td>"
27             . "<td>" . escape_html($e->{correo}) . "</td>"
28             . "<td>" . escape_html($e->{carrera}) . "</td>"
29             . "<td><a class=\"del\" href=\"/eliminar?id=\" . escape_html($e->{id})
30             . \">Eliminar</a></td>"
31             . "</tr>\n";
32     }
33     $filas = "<tr><td colspan=\"5\" class=\"vacio\">Aun no hay estudiantes registrados.</td></tr>"
34     >"
35     if $total == 0;
36
37     # --- Reporte 2: resumen por carrera (GROUP BY) ---
38     my $res = $dbh->prepare(
39         'SELECT carrera, COUNT(*) AS n FROM estudiantes GROUP BY carrera ORDER BY carrera');
40     $res->execute;
41     my $resumen = '';

```

crud_estudiantes.pl — parte 5/7

```

1  while (my $r = $res->fetchrow_hashref) {
2      $resumen .= "<li>" . escape_html($r->{carrera})
3                  . " <strong>" . escape_html($r->{n}) . "</strong></li>\n";
4  }
5  $resumen = "<li class=\"vacío\">Sin datos todavía.</li>" if $resumen eq '';
6
7  # --- HTML5 semántico ---
8  return <<"HTML";
9  <!DOCTYPE html>
10 <html lang="es-EC">
11 <head>
12   <meta charset="UTF-8">
13   <meta name="viewport" content="width=device-width, initial-scale=1">
14   <title>Registro de estudiantes - UTEQ</title>
15   <style>
16     /* ... reglas CSS (ver el archivo crud_estudiantes.pl) ... */
17   </style>
18 </head>
19 <body>
20
21   <header class="sitio">
22     <h1>Registro de estudiantes &middot; UTEQ</h1>
23     <p>Aplicación web en PERL conectada a PostgreSQL (appweb2026ppa)</p>
24   </header>
25
26   <main>
27     <!-- ALTA DE DATOS: formulario -->
28     <section class="alta">
29       <h2>Registrar estudiante</h2>
30       <form method="post" action="/agregar">
31         <label for="nombre">Nombre completo</label>
32         <input id="nombre" name="nombre" type="text" required minlength="3"
33               placeholder="Ej. Ana Perez" autofocus>
34
35         <label for="correo">Correo</label>
36         <input id="correo" name="correo" type="email" required
37               placeholder="ej. ana@uteq.edu.ec">
38
39         <label for="carrera">Carrera</label>
40         <select id="carrera" name="carrera" required>

```


crud_estudiantes.pl — parte 6/7

```

1      <option value=""-- selecciona --</option>
2      <option>Ingenieria de Software</option>
3      <option>Ingenieria en Computacion</option>
4      <option>Telematica</option>
5      </select>
6
7      <button type="submit">Guardar</button>
8  </form>
9  </section>
10
11  <!-- REPORTE: tabla de estudiantes + resumen por carrera -->
12  <section class="reporte">
13    <h2>Reporte de estudiantes ($total)</h2>
14    <table>
15      <thead>
16        <tr><th>ID</th><th>Nombre</th><th>Correo</th><th>Carrera</th><th></th></tr>
17      </thead>
18      <tbody>
19    $filas
20      </tbody>
21    </table>
22
23    <h2 style="margin-top:1.2rem">Resumen por carrera</h2>
24    <ul class="resumen">
25    $resumen
26      </ul>
27    </section>
28  </main>
29
30  <footer style="text-align:center;color:#6b7280;font-size:.8rem;padding:1rem">
31    UTEQ &middot; Aplicaciones Web &middot; Unidad II
32  </footer>
33 </body>
34 </html>
35 HTML
36 }
37
38 # Envia una respuesta 200 con el HTML.
39 sub responder {
40   my ($cli, $html) = @_;

```

crud_estudiantes.pl — parte 7/7

```

1  my $bytes = length $html;
2  print $cli "HTTP/1.1 200 OK\r\n";
3  print $cli "Content-Type: text/html; charset=utf-8\r\n";
4  print $cli "Content-Length: $bytes\r\n";
5  print $cli "Connection: close\r\n";
6  print $cli "\r\n";
7  print $cli $html;
8  }
9
10 # Envía una redirección 302 (patrón Post-Redirect-Get).
11 sub redirigir {
12     my ($cli, $destino) = @_;
13     print $cli "HTTP/1.1 302 Found\r\n";
14     print $cli "Location: $destino\r\n";
15     print $cli "Content-Length: 0\r\n";
16     print $cli "Connection: close\r\n";
17     print $cli "\r\n";
18 }
19
20 # Decodifica texto de URL: '+' -> espacio, '%XX' -> carácter.
21 sub url_decode {
22     my ($s) = @_;
23     $s =~ tr/+// /;
24     $s =~ s/%([0-9A-Fa-f]{2})/chr(hex($1))/ge;
25     return $s;
26 }
27
28 # Escapa < > & " para evitar XSS al mostrar datos del usuario.
29 sub escape_html {
30     my ($s) = @_;
31     $s = '' unless defined $s;
32     $s =~ s/&/&amp;/g;
33     $s =~ s/</&lt;/g;
34     $s =~ s/>/&gt;/g;
35     $s =~ s/"/&quot;/g;
36     return $s;
37 }

```

8.3 Estructura y flujo

El programa se organiza en: (1) conexión a la base, (2) creación de la tabla, (3) bucle del servidor que lee la petición y enruta según método y ruta, y (4) subrutinas auxiliares (parse_form, pagina, responder, redirigir, url_decode, escape_html). Cada dato que escribe el usuario se inserta con sentencias preparadas y se escapa al mostrarse.

8.4 Datos de prueba

Para poblar la tabla rápidamente desde psql o pgAdmin:

Datos de prueba

```
1 INSERT INTO estudiantes (nombre, correo, carrera) VALUES
2   ('Ana Perez',    'ana.perez@uteq.edu.ec',    'Ingenieria de Software'),
3   ('Luis Mora',    'luis.mora@uteq.edu.ec',    'Telematica'),
4   ('Carlos Vera',  'carlos.vera@uteq.edu.ec',  'Ingenieria en Computacion');
```

9 Aplicación con ORM: DBIx::Class (paso a paso)

Hasta aquí accedimos a la base con DBI y SQL escrito a mano. Un **ORM** (Object-Relational Mapping) da un paso más: representa cada tabla como una **clase** y cada fila como un **objeto**, de modo que trabajas con métodos de Perl en lugar de sentencias SQL. El ORM estándar de Perl es **DBIx::Class**, y puede incluso **crear las tablas a partir del modelo** con **deploy**.

9.1 ¿Cuándo usar un ORM?

- **A favor:** menos SQL repetitivo, manejo natural de relaciones (sin JOIN a mano) y portabilidad entre motores de base de datos.
- **En contra:** más dependencias y mayor curva de aprendizaje; para scripts pequeños, DBI suele ser más directo.
- **Regla práctica:** DBI para lo simple y puntual; ORM para aplicaciones con muchas entidades y relaciones.

9.2 Paso 1: Instalar los módulos

Además de `DBD::Pg`, el ORM necesita `DBIx::Class` y, para crear las tablas desde el modelo, `SQL::Translator`:

Instalar el ORM

```
1 C:\> cpan DBIx::Class SQL::Translator DBD::Pg
```

9.3 Paso 2: La estructura de archivos

El ORM organiza el modelo en una clase **Schema** y una clase **Result** por cada tabla. Crea esta estructura dentro del proyecto:

Estructura del proyecto

```
1 MiAppORM/
2 |- app_orm.pl           # programa principal
3 |- lib/
4   |- Escuela/
5     |- Schema.pm       # la clase Schema
6     |- Result/
7       |- Carrera.pm    # tabla 'carreras'
8       |- Estudiante.pm # tabla 'estudiantes'
```

9.4 Paso 3: La clase Schema

Es el punto central del ORM: carga automáticamente todas las clases Result de la carpeta **Result/**.

lib/Escuela/Schema.pm

```
1 package Escuela::Schema;
2 use strict;
3 use warnings;
4 use base 'DBIx::Class::Schema';
5
6 # Carga automaticamente las clases de lib/Escuela/Result/:
7 __PACKAGE__->load_namespaces;
8
9 1; # todo modulo Perl termina devolviendo un valor verdadero
```

9.5 Paso 4: Las clases Result (el modelo)

Cada Result describe una tabla: sus columnas, la clave primaria y sus **relaciones**. Aquí una carrera tiene muchos estudiantes (**has_many**) y cada estudiante pertenece a una carrera (**belongs_to**). Definir la relación es lo que permite navegar entre objetos sin escribir JOIN.

lib/Escuela/Result/Carrera.pm

```

1 package Escuela::Result::Carrera;
2 use strict;
3 use warnings;
4 use base 'DBIx::Class::Core';
5
6 __PACKAGE__->table('carreras');           # nombre de la tabla
7 __PACKAGE__->add_columns(
8     id      => { data_type => 'integer', is_auto_increment => 1 },
9     nombre => { data_type => 'varchar', size => 80, is_nullable => 0 },
10 );
11 __PACKAGE__->set_primary_key('id');
12 __PACKAGE__->add_unique_constraint(['nombre']);
13
14 # Una carrera tiene muchos estudiantes:
15 __PACKAGE__->has_many(
16     estudiantes => 'Escuela::Result::Estudiante', 'carrera_id'
17 );
18
19 1;

```

lib/Escuela/Result/Estudiante.pm

```

1 package Escuela::Result::Estudiante;
2 use strict;
3 use warnings;
4 use base 'DBIx::Class::Core';
5
6 __PACKAGE__->table('estudiantes');
7 __PACKAGE__->add_columns(
8     id          => { data_type => 'integer', is_auto_increment => 1 },
9     nombre      => { data_type => 'varchar', size => 100, is_nullable => 0 },
10    correo       => { data_type => 'varchar', size => 120, is_nullable => 0 },
11    carrera_id => { data_type => 'integer', is_nullable => 0 },
12 );
13 __PACKAGE__->set_primary_key('id');
14
15 # Cada estudiante pertenece a una carrera:
16 __PACKAGE__->belongs_to(
17     carrera => 'Escuela::Result::Carrera', 'carrera_id'
18 );
19
20 1;

```

9.6 Paso 5: Conectar y crear las tablas con el ORM

El método `connect` abre la conexión y `deploy` genera y ejecuta el SQL `CREATE TABLE` a partir del **modelo**: no escribimos SQL.

app_orm.pl — conexión y deploy

```
1  #!/usr/bin/perl
2  use strict;
3  use warnings;
4  use utf8;
5  use lib 'lib';           # para encontrar Escuela::Schema
6  use Escuela::Schema;
7
8  binmode(STDOUT, ':encoding(UTF-8)');
9
10 # Conexion (mismos datos; clave en comillas simples por el $ y la @):
11 my $dsn = "dbi:Pg:dbname=appweb2026ppa;host=127.0.0.1;port=5432";
12 my $schema = Escuela::Schema->connect(
13     $dsn, 'postgres', 'P@G$7Gr3$QL', { pg_enable_utf8 => 1 }
14 );
15
16 # Crear las tablas DESDE EL MODELO (el ORM genera el CREATE TABLE):
17 $schema->deploy;
18 print "Tablas creadas desde el modelo.\n";
```

Sobre deploy()

deploy crea las tablas la primera vez. Si ya existen, dará error; para recrearlas durante el desarrollo usa `deploy({ add_drop_table => 1 })`, que las elimina y vuelve a crear.

9.7 Paso 6: Operaciones CRUD con el ORM

Todas las operaciones se hacen con objetos y métodos, sin SQL. Observa cómo se navega la relación con `$e->carrera->nombre`:

app_orm.pl — CRUD

```

1  # CREATE: crear una carrera y un estudiante (sin escribir SQL)
2  my $soft = $schema->resultset('Carrera')
3      ->create({ nombre => 'Ingenieria de Software' });
4
5  $schema->resultset('Estudiante')->create({
6      nombre      => 'Ana Perez',
7      correo      => 'ana@uteq.edu.ec',
8      carrera_id => $soft->id,
9  });
10
11 # READ: listar estudiantes y su carrera (relacion, sin JOIN manual)
12 my $rs = $schema->resultset('Estudiante')
13     ->search({}, { order_by => 'me.id' });
14 print "Estudiantes:\n";
15 while (my $e = $rs->next) {
16     printf " [%d] %s - %s (%s)\n",
17         $e->id, $e->nombre, $e->correo, $e->carrera->nombre;
18 }
19
20 # UPDATE: cambiar el correo del estudiante con id 1
21 my $uno = $schema->resultset('Estudiante')->find(1);
22 $uno->update({ correo => 'ana.perez@uteq.edu.ec' }) if $uno;
23
24 # DELETE: borrar por condicion
25 $schema->resultset('Estudiante')
26     ->search({ nombre => 'Ana Perez' })->delete;

```

9.8 Paso 7: Integrarlo en la aplicación web

Para convertirlo en aplicación web, reutiliza el mini servidor de la sección anterior y reemplaza las llamadas DBI por las del ORM dentro del enrutamiento:

Integración en el servidor

```

1  # --- Alta (POST /agregar): crear con el ORM ---
2  $schema->resultset('Estudiante')->create({
3      nombre      => ${nombre},
4      correo      => ${correo},
5      carrera_id => ${carrera_id},
6  });
7
8  # --- Reporte (GET /): recorrer OBJETOS, no filas ---
9  my @todos = $schema->resultset('Estudiante')
10     ->search({}, { order_by => 'me.id' })->all;
11 for my $e (@todos) {
12     # usar $e->nombre, $e->correo, $e->carrera->nombre ...
13 }

```

9.9 Paso 8: Ejecutar y resultado

Ejecuta `app_orm.pl` con el botón Run (con PostgreSQL en marcha). La salida esperada en la consola:

Salida en la consola

```
1 Tablas creadas desde el modelo.
2 Estudiantes:
3 [1] Ana Perez - ana@uteq.edu.ec (Ingenieria de Software)
```

DBI vs ORM — en una frase

Con **DBI** controlas el SQL al detalle; con **DBIx::Class** ganas productividad y relaciones a cambio de más abstracción. Conocer ambos te hace un mejor desarrollador.

10 Seguridad en aplicaciones web

La seguridad no se «añade al final»: se diseña desde la primera línea. Estas consideraciones son transversales y reaparecerán en PHP, Java y .NET.

- **Validar la entrada en el servidor:** el cliente es manipulable; revalida siempre.
- **Escapar la salida:** neutraliza etiquetas como `<script>` (anti-XSS).
- **Sentencias preparadas:** marcadores `?` en toda consulta (anti-SQL Injection).
- **Credenciales:** claves con caracteres especiales en comillas simples; en producción, fuera del código (variables de entorno).
- **UTF-8 de extremo a extremo:** archivo, salida, base de datos y cabecera HTTP.

Estas prácticas cubren varias categorías del **OWASP Top 10** (en especial A03: Injection), el marco de referencia mundial de seguridad web.

11 Buenas prácticas y mantenimiento

- `use strict;` y `use warnings;` en todo script.
- Funciones pequeñas, cada una con una sola responsabilidad.
- Manejo de errores claro (`RaiseError`, or `die` con mensajes útiles).
- Separar la lógica de la presentación (plantillas en lugar de HTML incrustado, en proyectos grandes).
- Reutilizar con módulos (`.pm` en `lib/`) y probar (carpeta `t/`).
- Versionar con Git; nunca subir secretos al repositorio.
- Cerrar siempre lo que se abre: conexiones, sentencias y sockets.

12 Solución de problemas comunes

Síntoma	Causa y solución
<code>Can't locate DBD/Pg.pm</code>	Falta el módulo. Instálalo con <code>cpan</code> (p. ej. <code>cpan DBD::Pg</code>). El mini servidor no necesita <code>CGI.pm</code> .
El programa no hace nada	Es normal: un servidor se queda escuchando. Abre <code>http://localhost:8080</code> en el navegador.
<code>Address already in use</code>	El puerto 8080 está ocupado. Pulsa Stop, o cambia \$puerto a 8090.
<code>No reconoce perl.exe</code>	Revisa la sección 2.3: el intérprete debe apuntar a <code>C:\Strawberry\perl\bin\perl.exe</code> .
<code>connection refused</code>	PostgreSQL no está en ejecución, o el host/puerto son incorrectos.
<code>password authentication failed</code>	Clave incorrecta. Revisa que esté en comillas simples y bien escrita.
<code>database does not exist</code>	Crea la base: <code>CREATE DATABASE appweb2026ppa;</code> (<code>psql</code> o <code>pgAdmin</code>).
Tildes que salen mal	Activa UTF-8 en los cuatro puntos: archivo, salida, base de datos y cabecera HTTP.

13 Glosario de siglas y acrónimos

Sigla	Significado
PERL	Practical Extraction and Report Language.
CGI	Common Gateway Interface.
CPAN	Comprehensive Perl Archive Network.
IDE	Integrated Development Environment.
HTTP/HTTPS	HyperText Transfer Protocol / Secure.
HTML	HyperText Markup Language.
CSS	Cascading Style Sheets.

Sigla	Significado
URL	Uniform Resource Locator.
DBI	Database Interface (API de Perl para bases de datos).
DBD::Pg	Database Driver para PostgreSQL.
DSN	Data Source Name.
SQL	Structured Query Language.
CRUD	Create, Read, Update, Delete.
ORM	Object-Relational Mapping.
PSGI	Perl Web Server Gateway Interface.
XSS	Cross-Site Scripting.
PRG	Post-Redirect-Get.
UTF-8	Codificación de caracteres Unicode.
OWASP	Open Web Application Security Project.

14 Ejercicios propuestos

1. Añade a la aplicación «Hola Mundo» un segundo campo (por ejemplo, la edad) y muéstralo en el saludo.
2. Completa el CRUD agregando la operación **Actualizar**: un formulario para editar un estudiante existente.
3. Agrega validación adicional en el servidor: rechaza correos que no terminen en @uteq.edu.ec.
4. Crea una nueva tabla «materias» y un reporte que muestre las materias por carrera.
5. Reescribe la consulta del reporte para ordenar por nombre en vez de por id.
6. Investiga e implementa una versión mínima con el framework **Mojolicious** o **Dancer2** y compara la cantidad de código.

15 Recursos y referencias

- Documentación oficial de Perl: **perldoc.perl.org**

- CPAN (módulos): **metacpan.org**
- DBI: **metacpan.org/pod/DBI** · DBD::Pg: **metacpan.org/pod/DBD::Pg**
- PostgreSQL: **postgresql.org/docs**
- Strawberry Perl: **strawberryperl.com**
- OWASP Top 10: **owasp.org/Top10**